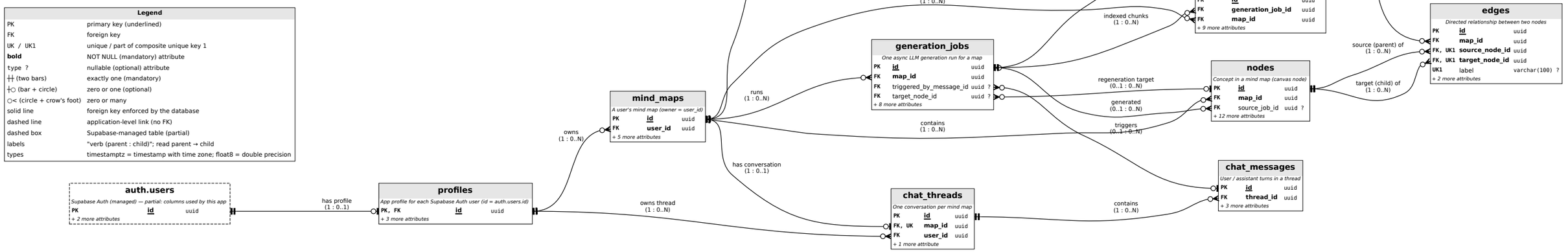
Overview — all entities, keys and relationships

Mind Map Generator — Entity-Relationship Diagram — overview (keys only)

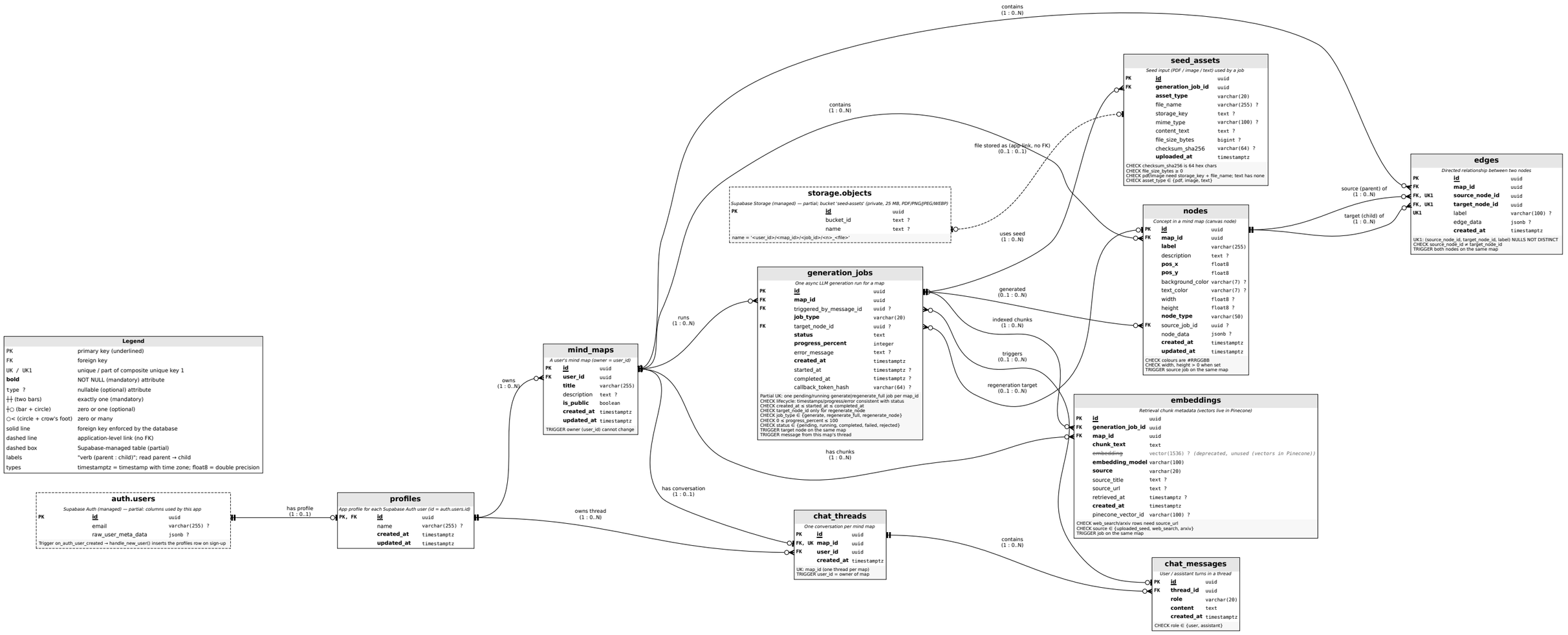
Final Supabase/PostgreSQL schema (supabase/migrations up to 20261015000000_sync_with_backend_contract.sql) · Crow's-foot notation



Complete diagram — all attributes and constraints

Mind Map Generator — Entity-Relationship Diagram (complete, all attributes)

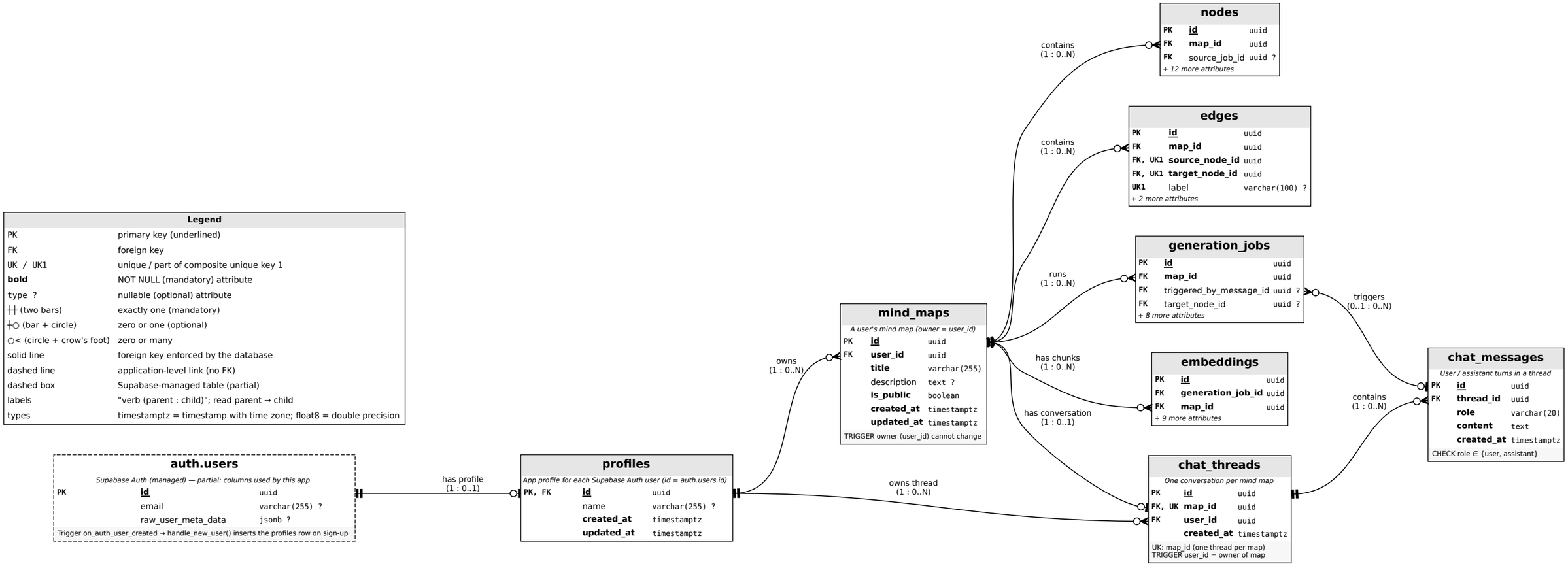
Final Supabase/PostgreSQL schema (supabase/migrations up to 20261015000000_sync_with_backend_contract.sql) · Crow's-foot notation



Section 1 — Users, mind maps and conversation

Mind Map Generator — Entity-Relationship Diagram: Section 1 — Users, mind maps and conversation

Final Supabase/PostgreSQL schema (supabase/migrations up to 20261015000000_sync_with_backend_contract.sql) · Crow's-foot notation



Section 2 — Generation jobs, seed input and retrieval chunks

Mind Map Generator — Entity-Relationship Diagram: Section 2 — Generation jobs, seed input and retrieval chunks

Final Supabase/PostgreSQL schema (supabase/migrations up to 20261015000000_sync_with_backend_contract.sql) · Crow's-foot notation

Legend	
PK	primary key (underlined)
FK	foreign key
UK / UK1	unique / part of composite unique key 1
bold	NOT NULL (mandatory) attribute
type ?	nullable (optional) attribute
┼┼ (two bars)	exactly one (mandatory)
┼○ (bar + circle)	zero or one (optional)
○< (circle + crow's foot)	zero or many
solid line	foreign key enforced by the database
dashed line	application-level link (no FK)
dashed box	Supabase-managed table (partial)
labels	"verb (parent : child)"; read parent → child
types	timestamptz = timestamp with time zone; float8 = double precision

storage.objects

Supabase Storage (managed) — partial; bucket 'seed-assets' (private, 25 MB, PDF/PNG/JPEG/WEBP)

PK	**id**	uuid
	bucket_id	text ?
	name	text ?

name = '<user_id>/<map_id>/<job_id>/<n>_<file>'

uses seed
(1 : 0..N)

file stored as (app link, no FK)
(0..1 : 0..1)

seed_assets

Seed input (PDF / image / text) used by a job

PK	**id**	uuid
FK	**generation_job_id**	uuid
	asset_type	varchar(20)
	file_name	varchar(255) ?
	storage_key	text ?
	mime_type	varchar(100) ?
	content_text	text ?
	file_size_bytes	bigint ?
	checksum_sha256	varchar(64) ?
	uploaded_at	timestamptz

CHECK checksum_sha256 is 64 hex chars
CHECK file_size_bytes ≥ 0
CHECK pdf/image need storage_key + file_name; text has none
CHECK asset_type ∈ {pdf, image, text}

indexed chunks
(1 : 0..N)

embeddings

Retrieval chunk metadata (vectors live in Pinecone)

PK	**id**	uuid
FK	**generation_job_id**	uuid
FK	**map_id**	uuid
	chunk_text	text
	~~embedding~~	vector(1536) ? *(deprecated, unused (vectors in Pinecone))*
	embedding_model	varchar(100)
	source	varchar(20)
	source_title	text ?
	source_url	text ?
	retrieved_at	timestamptz ?
	created_at	timestamptz
	pinecone_vector_id	varchar(100) ?

CHECK web_search/arxiv rows need source_url
CHECK source ∈ {uploaded_seed, web_search, arxiv}
TRIGGER job on the same map

has chunks
(1 : 0..N)

mind_maps

PK	**id**	uuid
FK	**user_id**	uuid

+ 5 more attributes

runs
(1 : 0..N)

generation_jobs

One async LLM generation run for a map

PK	**id**	uuid
FK	**map_id**	uuid
FK	triggered_by_message_id	uuid ?
	job_type	varchar(20)
FK	target_node_id	uuid ?
	status	text
	progress_percent	integer
	error_message	text ?
	created_at	timestamptz
	started_at	timestamptz ?
	completed_at	timestamptz ?
	callback_token_hash	varchar(64) ?

Partial UK: one pending/running generate|regenerate_full job per map_id
CHECK lifecycle: timestamps/progress/error consistent with status
CHECK created_at ≤ started_at ≤ completed_at
CHECK target_node_id only for regenerate_node
CHECK job_type ∈ {generate, regenerate_full, regenerate_node}
CHECK 0 ≤ progress_percent ≤ 100
CHECK status ∈ {pending, running, completed, failed, rejected}
TRIGGER target node on the same map
TRIGGER message from this map's thread

generated
(0..1 : 0..N)

regeneration target
(0..1 : 0..N)

triggers
(0..1 : 0..N)

nodes

PK	**id**	uuid
FK	**map_id**	uuid
FK	source_job_id	uuid ?

+ 12 more attributes

chat_messages

PK	**id**	uuid
FK	**thread_id**	uuid

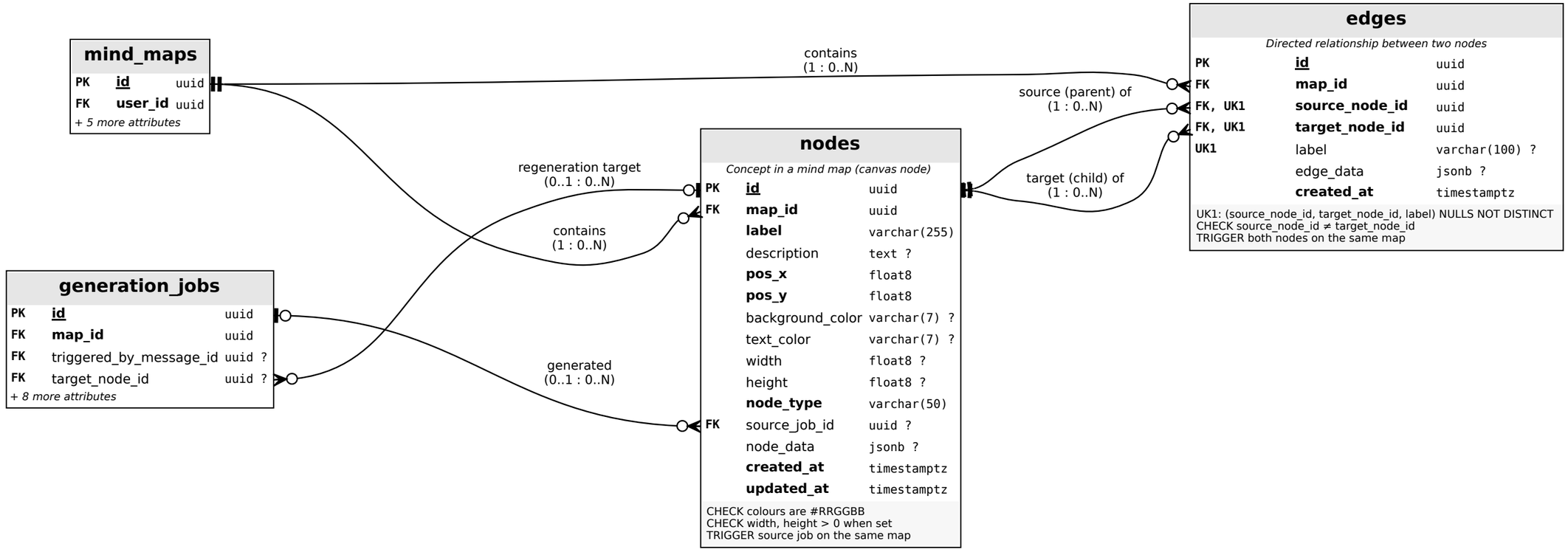
+ 3 more attributes

Section 3 — Mind-map graph (nodes and edges)

Mind Map Generator — Entity-Relationship Diagram: Section 3 — Mind-map graph (nodes and edges)

Final Supabase/PostgreSQL schema (supabase/migrations up to 20261015000000_sync_with_backend_contract.sql) · Crow's-foot notation

Legend	
PK	primary key (underlined)
FK	foreign key
UK / UK1	unique / part of composite unique key 1
bold	NOT NULL (mandatory) attribute
type ?	nullable (optional) attribute
┼┼ (two bars)	exactly one (mandatory)
┼○ (bar + circle)	zero or one (optional)
○< (circle + crow's foot)	zero or many
solid line	foreign key enforced by the database
dashed line	application-level link (no FK)
dashed box	Supabase-managed table (partial)
labels	"verb (parent : child)"; read parent → child
types	timestamptz = timestamp with time zone; float8 = double precision



Relationship explanation

Cardinality is read parent : child. Source: PostgreSQL catalog of a database built from `supabase/migrations`; validated against backend and LLM code.

#	Parent (one side)	Child	Foreign key	Cardinality (parent : child)	Type	Participation	ON DELETE	Enforced by	Meaning
1	auth.users	profiles	profiles.id → auth.users.id	1 : 0..1	one-to-one (optional child)	child mandatory; parent optional	CASCADE	Database FK	Every Supabase Auth user gets exactly one profile row with the same UUID. The FK allows at most one; the database trigger on_auth_user_created creates it on sign-up, so in practice it is 1:1.
2	profiles	mind_maps	mind_maps.user_id → profiles.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	A user owns any number of mind maps; every map has exactly one owner. Ownership cannot change (trigger trg_mind_maps_owner_immutable).
3	mind_maps	chat_threads	chat_threads.map_id → mind_maps.id	1 : 0..1	one-to-one (optional child)	child mandatory; parent optional	CASCADE	Database FK	A mind map has at most one chat thread (UNIQUE map_id); the backend creates it on the first generate/regenerate request.
4	profiles	chat_threads	chat_threads.user_id → profiles.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	Denormalised owner of a conversation. Trigger trg_chat_threads_owner_matches_map guarantees it equals the owner of the thread's map.
5	chat_threads	chat_messages	chat_messages.thread_id → chat_threads.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	A thread contains any number of user/assistant messages, ordered by created_at.
6	mind_maps	generation_jobs	generation_jobs.map_id → mind_maps.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	A map has any number of generation jobs (history). At most one generate/regenerate_full job can be pending/running per map (partial unique index).
7	chat_messages	generation_jobs	generation_jobs.triggered_by_message_id → chat_messages.id	0..1 : 0..N	one-to-many (optional parent)	child optional; parent optional	SET NULL	Database FK	The user chat message that started a job. Optional in the schema (SET NULL on delete); the backend always sets it for generate and regenerate jobs.
8	generation_jobs	seed_assets	seed_assets.generation_job_id → generation_jobs.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	The seed material (uploaded PDF/image text, or typed topic/ideas) a job generated from. Written by the backend when the LLM worker reports completion.
9	storage.objects	seed_assets	seed_assets.storage_key → storage.objects.name	0..1 : 0..1	one-to-one (optional both sides)	child optional; parent optional	—	Application only (no FK)	For pdf/image seed assets, storage_key holds the object path in the private 'seed-assets' bucket. Set by backend/uploads/service.py; there is no foreign key, and deleting a map does not delete the file. Text seeds have no file (CHECK chk_seed_asset_storage).
10	generation_jobs	embeddings	embeddings.generation_job_id → generation_jobs.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	Retrieval chunks recorded for a job. Table and repository helpers exist, but the current worker stores vectors only in Pinecone and no backend route writes this table.
11	mind_maps	embeddings	embeddings.map_id → mind_maps.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	Chunks belong to a map (Pinecone namespace = map_id). A trigger ensures the chunk's job belongs to the same map.
12	mind_maps	nodes	nodes.map_id → mind_maps.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	A mind map contains any number of nodes (concepts).
13	generation_jobs	nodes	nodes.source_job_id → generation_jobs.id	0..1 : 0..N	one-to-many (optional parent)	child optional; parent optional	SET NULL	Database FK	The job that produced a node. Optional: NULL for nodes added by hand on the canvas, or after the job is deleted (SET NULL). Trigger checks the job is on the same map.
14	nodes	generation_jobs	generation_jobs.target_node_id → nodes.id	0..1 : 0..N	one-to-many (optional parent)	child optional; parent optional	SET NULL (deferrable)	Database FK	For job_type = 'regenerate_node' only: the node being regenerated. Present in the schema, but the backend currently answers node regeneration with HTTP 501, so no such jobs are created.
15	mind_maps	edges	edges.map_id → mind_maps.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	A mind map contains any number of edges.
16	nodes	edges	edges.source_node_id → nodes.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	Edge start node. In generated maps this is the parent concept.
17	nodes	edges	edges.target_node_id → nodes.id	1 : 0..N	one-to-many	child mandatory; parent optional	CASCADE	Database FK	Edge end node. In generated maps this is the child concept.

Many-to-many and self-referencing

- **nodes ↔ nodes** — recursive many-to-many through the associative entity `edges` (two FKs to `nodes`). No table has a direct self-referencing FK.
- **generation_jobs ↔ nodes** — two independent one-to-many relationships (*generated*, *regeneration target*).

Enforced by the database

- **Primary keys** — every table has a single-column UUID primary key `id`. `profiles.id` is also the foreign key to `auth.users.id` (shared primary key, 1:1).

- **16 foreign keys** — all rows in the relationship table marked *Database FK*.
- **UNIQUE `chat_threads.map_id`** — makes mind map → chat thread one-to-one.
- **UNIQUE `edges (source_node_id, target_node_id, label)` NULLS NOT DISTINCT** — no duplicate edge between the same two nodes with the same label (two unlabelled duplicates also collide).
- **Partial UNIQUE `generation_jobs (map_id)` WHERE status ∈ {pending, running} AND job_type ∈ {generate, regenerate_full}** — at most one active full-map generation per map.
- **CHECK constraints** — allowed values (role, job_type, status, asset_type, source), the job lifecycle (status ↔ timestamps/progress/error), no self-loop edges, #RRGGBB colours, seed-asset storage rules, checksum format.
- **Triggers** — `on_auth_user_created` creates the profile on sign-up; same-map triggers on nodes, edges, generation_jobs and embeddings stop a row from pointing at a node/job/message of another map; `trg_chat_threads_owner_matches_map` keeps `chat_threads.user_id` equal to the map owner; `trg_mind_maps_owner_immutable` forbids changing a map's owner; `updated_at` triggers on profiles, mind_maps, nodes.

Inferred from application code (not enforced by the schema)

- **seed_assets.storage_key → storage.objects.name** — the uploaded PDF/image is stored at `<user_id>/<map_id>/<job_id>/<n>_<file>` in the private `seed-assets` bucket (`backend/uploads/service.py`). No FK; deleting a map does not delete the file.
- **embeddings.pinecone_vector_id → Pinecone** — vectors live in the Pinecone index (namespace = map_id), outside PostgreSQL, so Pinecone is not drawn as an entity.
- **Edges are directed parent → child, the map has a single root and no cycles** — enforced by the LLM worker (`llm/validation/schema.py`: single root, connected, acyclic), not by the database. The database only forbids self-loops and duplicates.
- **Every generate/regenerate job has a trigger message** — the backend always sets `triggered_by_message_id`, but the column is nullable in the schema.
- **Profile is effectively mandatory (1:1)** — the FK alone allows a user without a profile (0..1); the database trigger creates it at sign-up.
- **Schema present but not exercised by the current code** — `regenerate_node` jobs / `generation_jobs.target_node_id` (node regeneration returns HTTP 501); the `embeddings` table (repository helpers exist, but the worker writes vectors only to Pinecone); `mind_maps.description` and `is_public` (no route sets them yet); `nodes.width/height` and `edges.edge_data` (read by the API; set only by canvas edits).
- **Deprecated column** — `embeddings.embedding vector(1536)` is kept for existing data, is always NULL for rows written by the current backend, and can be dropped with `supabase/manual/drop_legacy_pgvector_column.sql`.

Security (RLS) — not part of the ER model

- Row Level Security is enabled on all nine tables; policies are not ER entities and are not drawn.
- Ownership has one source of truth, `mind_maps.user_id`; every child table proves ownership through its map.
- Through the Supabase Data API a user can read/write their own maps, nodes and edges, read their own chat, jobs, seed assets and embeddings, and read maps marked `is_public` (anonymous visitors too). `callback_token_hash` is never readable by API users.
- The FastAPI backend connects as the table owner / service role (bypasses RLS) and checks ownership itself (`assert_map_owner`). The LLM worker has no database access.